E-Commerce Customer Churn Analysis

MySQL Module End Assignment 2: Step-by-Step Solution Guide

Prepared by Aiswarya Krishna G

Part	What you will do
Step 0	Load the database and prepare MySQL Workbench
Part 1	Data cleaning: missing values, outliers, inconsistent labels (5 marks)
Part 2	Data transformation: rename, create and drop columns (3 marks)
Part 3	Data analysis: 17 questions plus the customer_returns table (17 marks)
Extra	Troubleshooting common errors

Before You Start

This guide walks through the whole assignment in the order it must be run: load the data, clean it, transform it, then answer the analysis questions. Every step has a short explanation of **what** it does and **why** it is written that way.

Step 0: Load the data and prepare Workbench

Open **E-Commerce_Customer_churn_db.sql** in MySQL Workbench and run the whole file. It drops and recreates the **ecomm** database, creates the **customer_churn** table (20 columns) and inserts all rows. The data deliberately contains NULLs, outliers and inconsistent labels (Phone, COD, CC) for you to fix.

```
USE ecomm;
SET SQL_SAFE_UPDATES = 0;   -- lets UPDATE/DELETE run without a key in the WHERE clause
SELECT COUNT(*) FROM customer_churn;   -- sanity check that the data loaded
```

Important: run cleaning and transformation code as one block (select it, then press Ctrl+Shift+Enter). Ctrl+Enter runs only ONE statement, and skipping a statement (for example the ADD COLUMN ChurnStatus) before the DROP COLUMN step destroys data you still need.

Part 1: Data Cleaning (5 marks)

1.1 Impute the mean

Columns: WarehouseToHome, HourSpendOnApp, OrderAmountHikeFromlastYear, DaySinceLastOrder. AVG() ignores NULLs, so it gives the mean of the known values. ROUND() converts it to the nearest integer. MySQL does not allow an UPDATE whose subquery reads the same table directly (error 1093), so the subquery is wrapped in a derived table **t**.

```
UPDATE customer_churn
SET WarehouseToHome = (SELECT ROUND(AVG(WarehouseToHome))
    FROM (SELECT WarehouseToHome FROM customer_churn) t)
WHERE WarehouseToHome IS NULL;

UPDATE customer_churn
SET HourSpendOnApp = (SELECT ROUND(AVG(HourSpendOnApp))
    FROM (SELECT HourSpendOnApp FROM customer_churn) t)
WHERE HourSpendOnApp IS NULL;

UPDATE customer_churn
SET OrderAmountHikeFromlastYear = (SELECT ROUND(AVG(OrderAmountHikeFromlastYear))
    FROM (SELECT OrderAmountHikeFromlastYear FROM customer_churn) t)
WHERE OrderAmountHikeFromlastYear IS NULL;

UPDATE customer_churn
SET DaySinceLastOrder = (SELECT ROUND(AVG(DaySinceLastOrder))
    FROM (SELECT DaySinceLastOrder FROM customer_churn) t)
WHERE DaySinceLastOrder IS NULL;
```

1.2 Impute the mode

Columns: Tenure, CouponUsed, OrderCount. MySQL has no MODE() function. The mode is the most frequent value, so we group by the value, count each group, sort from largest to smallest and keep the first row. The result is stored in a variable (@name) and used in the UPDATE.

```
SET @m_tenure = (SELECT Tenure FROM customer_churn WHERE Tenure IS NOT NULL
    GROUP BY Tenure ORDER BY COUNT(*) DESC LIMIT 1);
UPDATE customer_churn SET Tenure = @m_tenure WHERE Tenure IS NULL;

SET @m_coupon = (SELECT CouponUsed FROM customer_churn WHERE CouponUsed IS NOT NULL
    GROUP BY CouponUsed ORDER BY COUNT(*) DESC LIMIT 1);
UPDATE customer_churn SET CouponUsed = @m_coupon WHERE CouponUsed IS NULL;

SET @m_order = (SELECT OrderCount FROM customer_churn WHERE OrderCount IS NOT NULL
    GROUP BY OrderCount ORDER BY COUNT(*) DESC LIMIT 1);
UPDATE customer_churn SET OrderCount = @m_order WHERE OrderCount IS NULL;
```

1.3 Remove outliers

Rows where WarehouseToHome is greater than 100 km are unrealistic, so they are deleted. The steps follow the order given in the assignment (impute first, then delete).

```
DELETE FROM customer_churn WHERE WarehouseToHome > 100;
```

1.4 Fix inconsistencies

The same value written in different ways would split one group into several in later GROUP BY queries. We standardise the labels with UPDATE ... WHERE. Use exact matches (=) so only the intended values change.

```
UPDATE customer_churn SET PreferredLoginDevice = 'Mobile Phone'
WHERE PreferredLoginDevice = 'Phone';
```

```
UPDATE customer_churn SET PreferedOrderCat = 'Mobile Phone'
WHERE PreferedOrderCat = 'Mobile';

UPDATE customer_churn SET PreferredPaymentMode = 'Cash on Delivery'
WHERE PreferredPaymentMode = 'COD';

UPDATE customer_churn SET PreferredPaymentMode = 'Credit Card'
WHERE PreferredPaymentMode = 'CC';
```

Part 2: Data Transformation (3 marks)

2.1 Rename columns

RENAME COLUMN works on MySQL 8. On MySQL 5.7 use CHANGE old new datatype instead. From this point on, use the new names in every query.

```
ALTER TABLE customer_churn RENAME COLUMN PreferedOrderCat TO PreferredOrderCat;
ALTER TABLE customer_churn RENAME COLUMN HourSpendOnApp   TO HoursSpentOnApp;
```

2.2 Create the new columns

First add empty columns with ALTER TABLE, then fill them with UPDATE using IF(condition, value_if_true, value_if_false). Churn and Complain are BIT columns, so comparing with 1 works.

```
ALTER TABLE customer_churn ADD COLUMN ComplaintReceived VARCHAR(3);
ALTER TABLE customer_churn ADD COLUMN ChurnStatus VARCHAR(10);

UPDATE customer_churn SET ComplaintReceived = IF(Complain = 1, 'Yes', 'No');
UPDATE customer_churn SET ChurnStatus       = IF(Churn = 1, 'Churned', 'Active');
```

2.3 Drop the old columns

Only drop after the new columns are filled. Once dropped, the Churn and Complain data cannot be recovered except by reloading the original script.

```
ALTER TABLE customer_churn DROP COLUMN Churn, DROP COLUMN Complain;
```

Part 3: Data Exploration and Analysis (17 marks)

Each query below answers one question from the assignment. Aggregate functions (COUNT, SUM, AVG, MAX) summarise many rows into one value per group; GROUP BY defines the groups; ORDER BY ... DESC LIMIT 1 picks the top group; HAVING filters groups after aggregation.

Q1. Count of churned and active customers

Group by the new ChurnStatus column and count rows in each group.

```
SELECT ChurnStatus, COUNT(*) AS customers
FROM customer_churn
GROUP BY ChurnStatus;
```

Q2. Average tenure and total cashback of churned customers

Filter to churned rows with WHERE, then compute two aggregates in one SELECT.

```
SELECT AVG(Tenure) AS avg_tenure, SUM(CashbackAmount) AS total_cashback
FROM customer_churn
WHERE ChurnStatus = 'Churned';
```

Q3. Percentage of churned customers who complained

The test ComplaintReceived = 'Yes' returns 1 or 0, so SUM counts the complainers. Divide by the total and multiply by 100.

```
SELECT ROUND(100 * SUM(ComplaintReceived = 'Yes') / COUNT(*), 2) AS pct_complained
FROM customer_churn
WHERE ChurnStatus = 'Churned';
```

Q4. City tier with most churned customers who prefer Laptop & Accessory

Filter on both conditions, group by city tier, sort by the count and keep the top row.

```
SELECT CityTier, COUNT(*) AS churned_customers
FROM customer_churn
WHERE ChurnStatus = 'Churned' AND PreferredOrderCat = 'Laptop & Accessory'
GROUP BY CityTier
ORDER BY churned_customers DESC
LIMIT 1;
```

Q5. Most preferred payment mode among active customers

Same pattern: filter active customers, group by payment mode, take the largest group.

```
SELECT PreferredPaymentMode, COUNT(*) AS customers
FROM customer_churn
WHERE ChurnStatus = 'Active'
GROUP BY PreferredPaymentMode
ORDER BY customers DESC
LIMIT 1;
```

Q6. Total order amount hike for single customers who prefer Mobile Phone

Two filters joined with AND, then SUM of the hike column.

```
SELECT SUM(OrderAmountHikeFromlastYear) AS total_hike
FROM customer_churn
WHERE MaritalStatus = 'Single' AND PreferredOrderCat = 'Mobile Phone';
```

Q7. Average devices registered for UPI users

AVG over the rows where the payment mode is UPI.

```
SELECT AVG(NumberOfDeviceRegistered) AS avg_devices
FROM customer_churn
WHERE PreferredPaymentMode = 'UPI';
```

Q8. City tier with the highest number of customers

Count customers per tier and keep the largest.

```
SELECT CityTier, COUNT(*) AS customers
FROM customer_churn
GROUP BY CityTier
ORDER BY customers DESC
LIMIT 1;
```

Q9. Gender that used the most coupons

SUM of coupons per gender, largest first.

```
SELECT Gender, SUM(CouponUsed) AS total_coupons
FROM customer_churn
GROUP BY Gender
ORDER BY total_coupons DESC
LIMIT 1;
```

Q10. Customers and maximum hours on app per order category

Two aggregates for every category.

```
SELECT PreferredOrderCat, COUNT(*) AS customers, MAX(HoursSpentOnApp) AS max_hours
FROM customer_churn
GROUP BY PreferredOrderCat;
```

Q11. Total orders: Credit Card users with the maximum satisfaction score

A subquery finds the highest satisfaction score; the outer query filters on it and sums OrderCount.

```
SELECT SUM(OrderCount) AS total_orders
FROM customer_churn
WHERE PreferredPaymentMode = 'Credit Card'
  AND SatisfactionScore = (SELECT MAX(SatisfactionScore) FROM customer_churn);
```

Q12. Average satisfaction score of customers who complained

Filter on ComplaintReceived, then AVG.

```
SELECT AVG(SatisfactionScore) AS avg_satisfaction
FROM customer_churn
WHERE ComplaintReceived = 'Yes';
```

Q13. Preferred order categories of customers who used more than 5 coupons

DISTINCT removes repeats so each category is listed once.

```
SELECT DISTINCT PreferredOrderCat
FROM customer_churn
WHERE CouponUsed > 5;
```

Q14. Top 3 order categories by average cashback

Average cashback per category, sorted descending, limited to 3 rows.

```
SELECT PreferredOrderCat, AVG(CashbackAmount) AS avg_cashback
FROM customer_churn
GROUP BY PreferredOrderCat
ORDER BY avg_cashback DESC
LIMIT 3;
```

Q15. Payment modes with average tenure of 10 months and more than 500 orders

Conditions on aggregates must go in HAVING, not WHERE. Here 'more than 500 orders' is read as the total orders of that payment mode. If nothing returns, check AVG(Tenure) per mode; it may be a decimal such as 10.2.

```
SELECT PreferredPaymentMode
FROM customer_churn
GROUP BY PreferredPaymentMode
HAVING AVG(Tenure) = 10 AND SUM(OrderCount) > 500;
```

Q16. Distance categories and churn breakdown

CASE checks the WHEN conditions top to bottom, so each later branch only applies to values that failed the earlier ones. Grouping by the category and ChurnStatus gives the breakdown.

```
SELECT
  CASE
    WHEN WarehouseToHome <= 5  THEN 'Very Close Distance'
    WHEN WarehouseToHome <= 10 THEN 'Close Distance'
    WHEN WarehouseToHome <= 15 THEN 'Moderate Distance'
    ELSE 'Far Distance'
  END AS DistanceCategory,
  ChurnStatus,
  COUNT(*) AS customers
FROM customer_churn
GROUP BY DistanceCategory, ChurnStatus
ORDER BY DistanceCategory, ChurnStatus;
```

Q17. Married, City Tier 1 customers with above-average order count

The subquery returns one number (the overall average order count) which each row is compared with.

```
SELECT *
FROM customer_churn
WHERE MaritalStatus = 'Married' AND CityTier = 1
  AND OrderCount > (SELECT AVG(OrderCount) FROM customer_churn);
```

Q18 (a). Create the customer_returns table and insert the data

CREATE TABLE defines the structure (ReturnID is the primary key); INSERT adds the eight return records.

```
CREATE TABLE customer_returns (
  ReturnID     INT PRIMARY KEY,
  CustomerID   INT,
  ReturnDate   DATE,
  RefundAmount INT
);

INSERT INTO customer_returns (ReturnID, CustomerID, ReturnDate, RefundAmount) VALUES
(1001, 50022, '2023-01-01', 2130),
(1002, 50316, '2023-01-23', 2000),
(1003, 51099, '2023-02-14', 2290),
(1004, 52321, '2023-03-08', 2510),
(1005, 52928, '2023-03-20', 3000),
(1006, 53749, '2023-04-17', 1740),
```

```
(1007, 54206, '2023-04-21', 3250),
(1008, 54838, '2023-04-30', 1990);
```

Q18 (b). Return details of churned customers who complained

An INNER JOIN matches each return to its customer through CustomerID and keeps only customers present in both tables. The WHERE clause then keeps churned customers who complained. If a returned customer was deleted as an outlier in step 1.3, they will not appear.

```
SELECT r.*, c.*
FROM customer_returns r
JOIN customer_churn c ON r.CustomerID = c.CustomerID
WHERE c.ChurnStatus = 'Churned' AND c.ComplaintReceived = 'Yes';
```

Troubleshooting

- **Error 1054, Unknown column 'ChurnStatus':** the column was never created or the data was dropped before it was filled. Re-run the whole script from the top with Ctrl+Shift+Enter.
- **Error 1175 (safe update mode):** run SET SQL_SAFE_UPDATES = 0; first.
- **Error 1093 (can't specify target table):** wrap the subquery in a derived table, as in step 1.1.
- **Unknown column PreferedOrderCat / HourSpendOnApp:** those names only exist before step 2.1. After renaming, use PreferredOrderCat and HoursSpentOnApp.
- **Want to start over:** re-run the original .sql file; it rebuilds the database from scratch.